

C++ STL Cheatsheet

1. Vector

```
vector<int> v = {1, 2, 3};
v.push_back(4);
v.pop_back();
sort(v.begin(), v.end());
reverse(v.begin(), v.end());
int sum = accumulate(v.begin(), v.end(), 0);
```

2. Pair

```
pair<int, int> p = {1, 2};
cout << p.first << " " << p.second;

vector<pair<int, int>> vp = {{1, 2}, {3, 4}};
```

3. Set / Unordered Set

```
set<int> s;
s.insert(10);
s.erase(10);
bool exists = s.count(10);

unordered_set<int> us;
```

4. Map / Unordered Map

```
map<string, int> m;
m["apple"] = 2;
cout << m["apple"];

unordered_map<int, int> um;
for (auto& p : m) {
    cout << p.first << " " << p.second;
}
```

5. Stack

```
stack<int> st;
st.push(10);
st.pop();
cout << st.top();
```

C++ STL Cheatsheet

6. Queue / Deque

```
queue<int> q;  
q.push(1);  
q.pop();  
cout << q.front();
```

```
deque<int> dq;  
dq.push_front(1);  
dq.push_back(2);  
dq.pop_front();  
dq.pop_back();
```

7. Priority Queue (Heap)

```
priority_queue<int> pq; // Max heap
```

```
priority_queue<int, vector<int>, greater<int>> minpq; // Min heap
```

8. Algorithms

```
sort(v.begin(), v.end());  
reverse(v.begin(), v.end());  
auto it = find(v.begin(), v.end(), x);  
bool found = binary_search(v.begin(), v.end(), x);  
int mx = *max_element(v.begin(), v.end());  
int mn = *min_element(v.begin(), v.end());  
int freq = count(v.begin(), v.end(), x);
```

9. Useful Macros

```
#define pb push_back  
#define all(x) x.begin(), x.end()
```

10. STL Use Cases

Frequency count	=> unordered_map
Kth largest	=> priority_queue
Unique elements	=> set
Duplicate check	=> unordered_set
Bracket matching	=> stack<char>
BFS traversal	=> queue<int>
Sliding window max	=> deque<int>